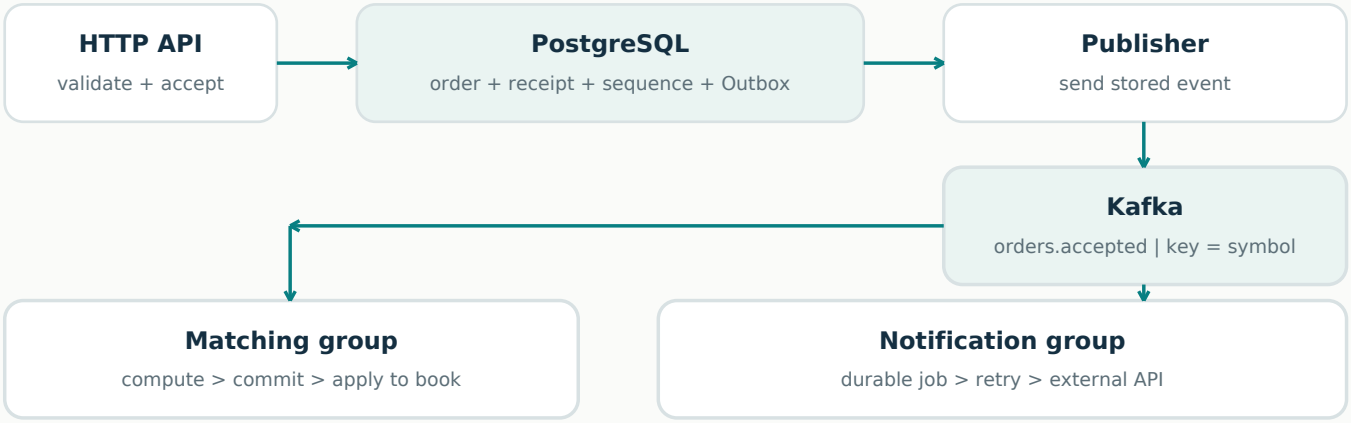
A reliable order flow, made reviewable.

What we achieved in the current Pooleno challenge project

An evidence-based record of the implementation, the failure cases it handles, and the engineering decisions behind it. Prepared on 1 November 2026.

ORDER CORRECTNESS	DURABLE DELIVERY	CLEAR REVIEW
Price priority, FIFO, maker pricing and partial fills	Transactional Outbox, Inbox and recoverable jobs	One-command verification, design notes and visual scenarios



The achievement

A client receives **202 only after durable acceptance**. Kafka delivery can repeat; trades and destination effects are protected by explicit idempotency boundaries. Matching and notification processing recover from durable records. [S2-S6]

Scope: only **pooleno-challenge**. Source: **main / 8776c88**, plus the observed uncommitted review-flow updates. This is a local challenge implementation, with production limits stated explicitly. It is not a production exchange or an end-to-end exactly-once system.

An honest API contract

The API separates durable acceptance from eventual execution. The response tells the client what has actually been committed, while GET reports the current matching result. [S2, S3]

Exact inputs and deliberate validation

BUY and SELL LIMIT orders use configured BTC-USD and ETH-USD markets. Prices and quantities enter as decimal strings, become exact bigint units, and are stored without passing through floating-point arithmetic. Excess precision, invalid increments, zero values, unsupported symbols, numeric amounts and unknown body properties are rejected.

One transaction closes the acceptance gap

The order, stored idempotency receipt, per-symbol sequence and Outbox event commit together. Failure leaves no partial acceptance or advanced sequence. Kafka is outside the request path, so a broker outage delays processing while the API can still accept safely.

Client situation	Behavior and value
New valid request	202 + orderId + QUEUED receipt + status URL, after commit.
Same key, same meaning	Replay the original receipt; equivalent decimal spellings are canonicalized.
Same key, changed meaning	422; the original order and receipt remain intact.
Same key still in progress	409 with retry advice after a bounded lock wait.
Infrastructure outcome uncertain	503 with advice to retry the same key; the retry resolves the outcome.
Read the accepted order	GET returns durable order state and trades in execution order.

Why this is worth highlighting

Idempotency is enforced by database transactions and uniqueness, including concurrent requests. It is not an in-memory duplicate cache. A lost HTTP response does not require the client to guess whether to create another order.

A precise guarantee

202 means durable, not filled. QUEUED, OPEN, PARTIALLY_FILLED and FILLED are distinct states. Validation precedes acceptance; a dependency outage delays accepted work instead of changing an order into a business rejection.

Evidence: acceptance.test.ts covers concurrent duplicates, rollback, bounded contention and equivalent decimals. crash-recovery.test.ts covers death after commit but before the response. [S3, S7]

Correctness with explainable costs

The matching engine stays pure TypeScript. It matches best price first, then oldest order within a price level, executes at the resting maker price, and preserves partial remainders. [S4]

Structure	Purpose
B+ tree: sorted-btree	Ordered bigint price levels; lookup, insertion and deletion in O(log P).
Singly linked FIFO per price	O(1) append and removal from the head, without shifting an unbounded array.
Map of symbol to book	Independent books; exact integer quantities and prices throughout.

The working fixture

Resting sells arrive as S1: 2 at 100, S2: 3 at 100, then S3: 1 at 99. A BUY for 4 at limit 100 executes in the following order. This is the executable demo fixture. [S8]

Step	Maker	Quantity	Price	Reason
1	S3	1	99	Best price wins
2	S1	2	100	Oldest at this price
3	S2	1	100	Partial fill; 2 remain

The cost model

P = price levels; **K** = makers touched; **L** = levels emptied; **N** = resting orders. Computing a match costs **O(log P + K + L)**. Applying it costs **O(log P + K + L log P)**. Rebuilding costs **O(N log P)** plus reading N rows; memory is **O(N + P)**. Database work also grows with the makers touched.

Commit before changing the book

Compute without mutating RAM; commit the Inbox record, quantities, trades and progress together; then apply the committed result to the book. Only then may the Kafka offset advance. On uncertainty, discard the cache and rebuild committed open quantities in acceptance order.

Evidence beyond examples

An independent sorted-list reference matcher checks every trade and resulting book over **15,000 seeded random orders**, including quantity conservation and non-negative remainders. SELL symmetry, non-crossing orders, empty levels and exact crossing boundaries have dedicated tests. [S7]

Ordering without hiding the crash windows

Direct Kafka integration keeps publication, partition ordering, offset commits and consumer recovery explicit, even after adding NestJS. The guarantee is at-least-once delivery with idempotent effects. [S2, S5]

The durable publication path

The publisher reads stored Outbox events, sends only the earliest unpublished event for each symbol, and marks publication after Kafka acknowledges. It uses an idempotent producer and the symbol as the Kafka key. If the process dies after sending but before marking, it resends the same event. A failed symbol holds back its own later publications.

Offsets follow durable work

Auto-commit is disabled. Each group commits offset + 1 after its own durable handling. Matching and notification groups progress independently. A replayed matching event is absorbed by the transactional Inbox; a replayed notification event creates no second job. A consumer does not commit after losing its partitions.

Improvement achieved	Why it matters
A sequential lane per partition	An asynchronous database stall or repeatable failure in BTC does not immediately block ETH on a different partition. Each partition retains its own order.
Per-symbol sequence guard	Missing, unknown old or out-of-order events cannot silently change the book. A gap holds processing for operator recovery.
Durable quarantine	Malformed or unreadable events are recorded with their position before skipping; a failed quarantine write prevents offset advancement.
Stored source for repair	The operations runbook explains republishing unapplied Outbox events after a lost Kafka event.

The partition bottleneck we addressed

The move to independent partition lanes removed unnecessary waiting behind another partition's database work. Tests deliberately hold BTC writes while ETH finishes, then verify BTC resumes in order. Malformed events also have real-consumer tests. [S7]

Boundaries we kept visible

Lanes share one process and one incoming stream. A full lane can backpressure all partitions; CPU-heavy synchronous matching still shares the Node.js thread. Single ownership is not horizontal matching scale. Same-symbol work remains sequential. [S2, S5]

Retries that survive the worker

Notification intake and delivery are separate durable steps. The external consumer records its Inbox entry and job before committing the Kafka offset; the worker later makes the HTTP call. [S6]

Failure isolation and durable intent

A destination outage does not stop matching or notification intake. A Kafka outage does not prevent delivery of already stored jobs. Attempts, next due time, terminal state and errors survive process restart. PENDING, SUCCEEDED and FAILED have explicit meanings; scheduling work never counts as successful delivery.

Outcome	Implemented handling
Network error, timeout or transient status	Retry using persisted exponential delay with jitter. Honor Retry-After within the configured cap.
Permanent rejection	Mark FAILED immediately; retain the failure for inspection and explicit redrive.
Retry budget exhausted	Mark FAILED. Redrive reuses the event and destination idempotency key.
Destination committed; response lost	Repeat the same bytes under external-notify:<eventId>; the destination replays its stored result.
Late result after job changed	Compare-and-set prevents an old attempt from overwriting newer job state.

The destination proves its own side

The independent mock API imports no backend code and uses its own durable SQLite transaction to save the idempotency record and effect together. It can commit an effect, lose the response, restart, and still replay without applying the effect twice. Tests exercise that remote-success/local-uncertainty window.

Bounded operational defaults

The configured defaults are a 5-second request timeout, a 1-second initial retry delay, a 5-minute delay cap and 400 recorded attempts. Jitter spreads retries. A crash before recording the outcome consumes no recorded attempt, so a new owner can reclaim the still-pending job. [S6]

The essential distributed-systems lesson

The local Inbox cannot resolve a lost remote response. **Destination-owned idempotency** closes that gap, for as long as the destination retains the key. The whole workflow must not be described as exactly once.

Designed to stop and start safely

Lifecycle behavior is implemented in the application. Kubernetes deployment considerations are documented; no Kubernetes cluster deployment or manifests are claimed. [S2, S9]

Graceful shutdown. SIGTERM or SIGINT makes the role unready, stops new intake, drains in-flight work and closes resources. The configured deadline bounds shutdown; a second signal exits immediately.

Crash recovery. A forced exit leaves durable state to drive replay. Matching restores committed orders, FIFO and remainders. An ambiguous commit triggers a rebuild instead of trusting RAM.

Single ownership. Each worker holds a PostgreSQL session advisory lock outside its application context. A standby waits; ownership loss stops the active role. The lock is released after draining and cleanup.

Role	Readiness reflects
API	A usable database pool. Kafka is not required for durable acceptance.
Publisher	Ownership and broker availability.
Matching	Ownership, restored books, assigned partitions and no held failing event.
External	Ownership and assigned partitions. A destination outage does not make the role unready.

Liveness is separate and does not depend on external services. GET /ops/backlog reports Outbox, matching, notification and Kafka lag information, including a bounded response when Kafka is unavailable. Logs connect requestId, orderId, eventId, symbol and sequence across roles. [S9]

Container and deployment work completed

Docker Compose runs the local stack. The production image uses a non-root user and starts Node directly. Configuration comes from the environment. The design explains probe mapping, termination grace, startup jobs, binding addresses, restricted operational routes and stronger broker replication for a real deployment.

What the deployment review still leaves open

A replacement worker waits for the existing owner's lock. A prior review identified a possible default rolling-update stall; it was not reproduced in a cluster. The worker update strategy, resource budgets and secret delivery still need deployment-specific decisions. Advisory locks and guards do not provide distributed fencing. [S10]

Proof, with its date and limits

The project uses unit tests, real-database integration tests, real Kafka/process tests and a restart proof. The evidence below separates fresh checks from earlier recorded runs. [S7, S8]

Evidence	Result	Status for this PDF
Backend typecheck, lint and build	All passed	Freshly run at 8776c88
Backend unit suite	211 tests / 10 files passed	Freshly run at 8776c88
Viewer unit suite and production build	41 tests / 4 files; build passed	Freshly run at 8776c88
Full backend run (documented)	211 unit + 221 integration/e2e tests	Separate recorded 1 Nov full run; not rerun here
Walkthrough and crash proof	14/14 + 9/9 checks; replay unchanged	Same earlier documented run
Viewer browser coverage	21 browser tests passed	Prior viewer-flow report; not rerun

The valuable failures under test

Boundary	What the tests assert after failure
HTTP acceptance	Commit-before-response death returns the original receipt; a partial write rolls back without a sequence gap.
Outbox and Kafka	Send-before-mark death resends stored bytes without a repeated business effect.
Matching and offsets	Commit-before-offset death adds no trade; uncertain commits rebuild the correct book.
Independent partitions	A held or failing BTC transaction allows ETH on another partition to finish.
External response	A committed remote effect plus lost response still produces one destination effect, including after restart.
Ownership and shutdown	Lost locks, rebalance, stuck dependencies and deadlines do not advance offsets beyond durable work.

Verification limit: this PDF task did not rerun the PostgreSQL/Kafka integration suite, browser suite, container build or full review.sh. The earlier aggregate counts are historical evidence, not a fresh total for the current commit. No latency percentiles, production load certification or cluster behavior are established by these counts.

We made the decisions visible

A reviewer can run the real backend checks, then use an optional visual walkthrough to understand the same decisions. The viewer is independent of the backend and remains outside the challenge runtime. [S8, S11]

One command to inspect the real system

./review.sh builds images, starts the stack, exercises acceptance and matching, kills real processes, restarts infrastructure and checks that quantities, queue order, trades and destination effects remain correct. It repeats the walkthrough with the same keys and opens the scenario viewer at **127.0.0.1:8084**. **./review.sh --full** adds typechecking, lint and the full PostgreSQL/Kafka test suite.

Operational detail: review.sh first removes this Compose project's containers and volumes. It is a reset-and-review workflow. That behavior is documented; it should be used with disposable review data.

Six primary scenarios, plus focused variants

The viewer covers matching, non-crossing orders, identical requests, changed-key payloads, a lost external response, and acceptance during a Kafka outage. Additional cases show SELL symmetry, partial takers, commit/crash/replay, concurrent requests, invalid precision, zero quantity, transient failure, permanent rejection, redrive and exhausted retries.

Improvement made during our flow	Reviewer benefit
One synchronized timeline	Every journey starts with a named request and explicit resting-book preconditions. No separate internal clock creates unexplained jumps.
Expandable component internals	Order API, Matching, publisher and notification worker explain what happens, why, and where it lives in code.
Distinct visual states and paths	Current, next and parallel work differ; request and response directions are visibly separate.
Zoom, pan and keyboard controls	The canvas fits different displays; playback, replay, speed and reduced-motion handling support inspection.
Challenge relevance and evidence	Each case links the requirement, implementation choice and backend test references. Extensions are labeled.

A deliberate presentation boundary

The viewer uses **authored example frames**. It submits no orders, injects no live faults and proves no backend guarantee by itself. Executable backend tests provide the proof; the viewer makes the reasoning easier to inspect.

Framework familiarity, explicit fundamentals

The current project uses NestJS with the Fastify adapter and keeps the direct Kafka client. This aligns the implementation with the role's NestJS requirement while preserving visible messaging decisions. [S12]

What Nest owns	What remains plain TypeScript
Controllers, modules and dependency wiring	Matching engine, order books and price levels
Worker application contexts and resource cleanup	SQL repositories and reusable business functions
Explicit injection tokens and factory/value providers	Kafka clients, consumer loops and partition lanes

The migration preserved the important behavior

Fastify continues to validate and serialize through the existing TypeBox schemas. Explicit injection tokens work across development, compilation and tests without emitted decorator metadata. Adapter settings and the exception filter preserve API behavior. Worker ownership encloses the Nest context so cleanup finishes before a successor can take over.

The engineering choices worth presenting

A transactional Outbox over a dual write. One database commit protects accepted work without adding CDC infrastructure.

A rebuildable book over a second source of truth. RAM accelerates computation; PostgreSQL decides what survived.

Durable jobs over sleeping in a Kafka handler. Destination retries do not hold up matching or Kafka intake.

One active owner over premature distributed matching. The implementation stays explainable while documenting the fencing required for real multi-owner scale.

A narrow domain over exchange feature sprawl. LIMIT matching, failure recovery and clear evidence receive the attention the assignment asks for.

What this demonstrates

Strong evidence of backend reliability judgment: exact arithmetic, transactional consistency, explicit replay behavior, failure isolation, testable pure logic and operational explanations. It supports a **Senior Node.js backend** narrative; it does not independently establish production operating experience or deep runtime performance specialization.

Useful explanation sequence: why 202 is safe; where duplicates can arise; why each offset can advance; what is reconstructed after a crash; which limit would justify the next optimization.

A complete story with honest boundaries

The explicit assignment asks for order matching, Kafka processing, failure handling, complexity and system-design explanations, tests, and a simple local setup. PostgreSQL, 202, request idempotency, Outbox/Inbox and the optional viewer are implementation choices serving that goal. [S1]

What remains outside the achievement claim

No production authentication, wallets, settlement, market orders, cancel/amend, distributed matching ownership, Kubernetes manifests or cluster validation. No full fencing, automated retention, alerting stack or metrics endpoint. The local setup uses one PostgreSQL and one Kafka broker. Notification effects depend on destination key retention; notifications need not wait for matching or preserve notification order. [S2]

Evidence index

[S1] Assignment. /home/kiyanof/Downloads/57384601.pdf; one-page assignment and evaluation focus.

[S2] Architecture. docs/architecture.md; contracts, complexity, scaling, Kubernetes and limits.

[S3] Acceptance. docs/api.md; src/modules/orders/accept-order.ts; tests/integration/acceptance.test.ts.

[S4] Matching. src/modules/matching/{matching-engine,order-book,price-level,apply-accepted-order,restore-book}.ts.

[S5] Messaging. src/modules/messaging/{publish-outbox,consumer-session,partition-lanes}.ts; docs/adr/0002-messaging-and-matching.md.

[S6] Delivery. docs/adr/0003-external-notifications.md; src/modules/external/; mock-api/; tests/integration/external-retry.test.ts.

[S7] Tests. docs/testing.md; tests/unit/matching.test.ts; tests/integration/matching-recovery.test.ts; tests/e2e/{crash-recovery,partition-lanes,lifecycle}.test.ts.

[S8] Review workflow. README.md; review.sh; scripts/demo.ts; scripts/crash-proof.ts.

[S9] Operations. docs/operations.md; docs/adr/0004-shutdown-health-and-visibility.md; src/app/; Dockerfile; compose.yaml.

[S10] Deployment assessment. Chat: Clarify Kubernetes suitability requirements; 1 Nov 2026, final code-coverage review. Rollout concern is an untested inference.

[S11] Visual review. extras/README.md; extras/frontend/; chat: Research distinctive code challenge...; final walkthrough-variants report, 1 Nov 2026.

[S12] NestJS integration. docs/adr/0005-nestjs-on-fastify.md; src/app/; current package.json; commit 3185ac3.

Paths are relative to /home/kiyanof/Projects/pooleno-challenge unless absolute. Base commit: 8776c88041a2c06dcec1cf336821ed7804d69dcb, plus observed uncommitted review-flow updates. Fresh checks were run on 1 November 2026. No application source changes were made to prepare this report.